BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN MÔN HỌC
CƠ SỞ DỮ LIỆU NÂNG CAO

TÌM HIỂU PHÂN MẢNH HỖN HỢP TRONG CSDL PHÂN TÁN VÀ ỨNG DỤNG CHO QUẢN LÝ ĐƠN HÀNG

Ngành:	CÔNG NGHỆ THÔNG TIN	
Giảng viên hướng dẫn:	ThS. Nguyễn Văn Danh	
Sinh viên thực hiện:	Trần Gia Thịnh	MSSV: 2380602136
	Phạm Minh Triết	MSSV: 2380605829
Lớp:	23DTHE3	

TP. Hồ Chí Minh, tháng 10 năm 2026

LỜI CẢM ƠN

Để hoàn thành đồ án môn học Cơ Sở Dữ Liệu Nâng Cao, nhóm chúng em xin gửi lời cảm ơn chân thành đến thầy ThS. Nguyễn Văn Danh, giảng viên hướng dẫn. Trong suốt quá trình học tập và thực hiện đề tài, thầy đã tận tình giảng dạy, định hướng và góp ý, giúp nhóm hiểu rõ hơn các vấn đề trong thiết kế cơ sở dữ liệu phân tán.

Nhóm cũng xin cảm ơn Khoa Công nghệ Thông tin, Trường Đại học Công nghệ TP. Hồ Chí Minh đã tạo điều kiện về môi trường học tập và tài liệu tham khảo.

Do thời gian và kiến thức còn hạn chế, đồ án khó tránh khỏi thiếu sót. Nhóm rất mong nhận được ý kiến đóng góp của thầy để hoàn thiện hơn.

Nhóm sinh viên thực hiện: Trần Gia Thịnh – Phạm Minh Triết

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TP. Hồ Chí Minh, ngày ..... tháng ..... năm 2026

Giảng viên hướng dẫn

BẢNG PHÂN CÔNG CÔNG VIỆC

Phân công công việc trong nhóm

STT	Họ và tên	MSSV	Nhiệm vụ	Mức độ hoàn thành
1	Trần Gia Thịnh	2380602136	Tìm hiểu tổng quan CSDL phân tán, phân mảnh ngang, phân mảnh dọc và phân mảnh hỗn hợp (Chương 1, 2, 3); dựng slide và thuyết trình phần lý thuyết	100%
2	Phạm Minh Triết	2380605829	Phân tích bài toán quản lý đơn hàng, thiết kế phân mảnh và cấp phát, cài đặt thực nghiệm trên SQL Server (Chương 4); dựng slide và thuyết trình phần ứng dụng	100%
	Cả nhóm		Lời mở đầu, kết luận (Chương 5), rà soát và hoàn thiện báo cáo	100%

MỤC LỤC

LỜI MỞ ĐẦU 1
Lý do chọn đề tài 1
Mục tiêu đề tài 1
CHƯƠNG 1. TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU PHÂN TÁN 2
1.1 Khái niệm cơ sở dữ liệu phân tán 2
1.2 So sánh CSDL tập trung và CSDL phân tán 2
1.3 Kiến trúc tham chiếu của CSDL phân tán 2
1.4 Tính trong suốt trong CSDL phân tán 3
1.5 Thiết kế cơ sở dữ liệu phân tán 3
CHƯƠNG 2. PHÂN MẢNH DỮ LIỆU TRONG CSDL PHÂN TÁN 4
2.1 Lý do và đơn vị phân mảnh 4
2.2 Các quy tắc đúng đắn của phân mảnh 4
2.3 Phân mảnh ngang nguyên thủy 5
2.4 Phân mảnh ngang dẫn xuất 6
2.5 Phân mảnh dọc 6
2.6 Cấp phát và nhân bản 7
CHƯƠNG 3. PHÂN MẢNH HỖN HỢP 8
3.1 Định nghĩa và động cơ sử dụng 8
3.2 Chiến lược ngang trước, dọc sau (HV) 8
3.3 Chiến lược dọc trước, ngang sau (VH) 9
3.4 Cây phân mảnh và tái thiết quan hệ toàn cục 10
3.5 Kiểm tra tính đúng đắn 11
3.6 Ảnh hưởng đến xử lý truy vấn và giao dịch 11
3.7 Ưu điểm, hạn chế và so sánh 12
CHƯƠNG 4. ỨNG DỤNG PHÂN MẢNH HỖN HỢP CHO HỆ THỐNG QUẢN LÝ ĐƠN HÀNG 13
4.1 Mô tả bài toán 13
4.2 Phân tích ứng dụng 13

4.3 Thiết kế phân mảnh 14
4.4 Lược đồ cấp phát 16
4.5 Cài đặt thực nghiệm trên SQL Server 16
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 19
5.1 Kết quả đạt được.....19
5.2 Hạn chế 19
5.3 Hướng phát triển.....19
TÀI LIỆU THAM KHẢO 20

DANH MỤC TỪ VIẾT TẮT

Danh mục từ viết tắt

Từ viết tắt	Tiếng Anh	Tiếng Việt
CSDL	Database	Cơ sở dữ liệu
CSDLPT	Distributed Database	Cơ sở dữ liệu phân tán
DBMS	Database Management System	Hệ quản trị cơ sở dữ liệu
DDBMS	Distributed Database Management System	Hệ quản trị cơ sở dữ liệu phân tán
HV	Horizontal – Vertical	Phân mảnh hỗn hợp ngang trước, dọc sau
VH	Vertical – Horizontal	Phân mảnh hỗn hợp dọc trước, ngang sau
BEA	Bond Energy Algorithm	Thuật toán năng lượng liên kết
AA	Attribute Affinity matrix	Ma trận ái lực thuộc tính
CA	Clustered Affinity matrix	Ma trận ái lực đã gom cụm
PK	Primary Key	Khóa chính
2PC	Two-Phase Commit	Giao thức cam kết hai pha
MSDTC	Microsoft Distributed Transaction Coordinator	Bộ điều phối giao dịch phân tán của Microsoft
SSMS	SQL Server Management Studio	Công cụ quản trị SQL Server

DANH MỤC HÌNH

Hình 2.1: *Minh họa ba kiểu phân mảnh* 4

Hình 3.1: *Cây phân mảnh hỗn hợp HV của quan hệ KhachHang* 9

Hình 3.2: *Cây phân mảnh hỗn hợp VH của quan hệ NhanVien*.....10

Hình 4.1: *Kết quả chạy phần kiểm tra trong 03_reconstruct_check.sql: 14/14 phép kiểm tra ĐẠT* 17

Hình 4.2: *Giao dịch phân tán: công nợ KH007 trước và sau khi lập đơn DH046; đơn vượt hạn mức của KH022 bị hủy với lỗi 50002 và không site nào bị ghi dở*.....18

DANH MỤC BẢNG

Bảng 1.1: *So sánh CSDL tập trung và CSDL phân tán* 2

Bảng 2.1: *Ma trận sử dụng thuộc tính use(q_i, A_j)* 7

Bảng 3.1: *Nhu cầu truy cập KhachHang theo dòng và theo cột* 8

Bảng 3.2: *So sánh ba kiểu phân mảnh*.....12

Bảng 4.1: *Các ứng dụng chính của hệ thống*.....14

Bảng 4.2: *Kiểu phân mảnh của từng quan hệ*.....14

Bảng 4.3: *Lược đồ cấp phát*.....16

LỜI MỞ ĐẦU

Lý do chọn đề tài

Các doanh nghiệp hiện nay thường hoạt động trên nhiều địa điểm: trụ sở chính, chi nhánh ở nhiều tỉnh thành, kho hàng, cửa hàng. Nếu toàn bộ dữ liệu đặt tập trung tại một máy chủ, mọi giao dịch ở chi nhánh đều phải gửi về trung tâm. Điều đó làm tăng thời gian phản hồi và tải trên đường truyền, và cả hệ thống ngừng hoạt động khi máy chủ trung tâm gặp sự cố. Cơ sở dữ liệu phân tán giải quyết vấn đề này bằng cách đặt dữ liệu tại nhiều site, gần nơi dữ liệu được sử dụng, nhưng vẫn cho người dùng thấy như một cơ sở dữ liệu thống nhất.

Vấn đề cốt lõi khi thiết kế cơ sở dữ liệu phân tán là **phân mảnh**: chia mỗi quan hệ thành những phần nhỏ hơn để đặt ở các site khác nhau. Phân mảnh ngang chia theo dòng, phân mảnh dọc chia theo cột. Tuy nhiên, trong thực tế, các nhóm người dùng thường vừa chỉ quan tâm đến một số dòng, vừa chỉ quan tâm đến một số cột. Khi đó, chỉ dùng một kiểu phân mảnh là không đủ, và **phân mảnh hỗn hợp**, tức là kết hợp cả hai kiểu, là lựa chọn phù hợp.

Bài toán quản lý đơn hàng của một công ty có nhiều chi nhánh là ví dụ điển hình. Chi nhánh chỉ làm việc với khách hàng và đơn hàng của mình. Phòng kế toán ở trụ sở cần thông tin công nợ của khách hàng ở mọi chi nhánh. Phòng nhân sự quản lý dữ liệu lương nhạy cảm, không được để lộ ra chi nhánh. Vì vậy nhóm chọn đề tài **"Tìm hiểu Phân mảnh hỗn hợp trong CSDL Phân tán và ứng dụng cho quản lý đơn hàng"** để vừa nắm vững lý thuyết, vừa thấy được cách áp dụng vào một bài toán cụ thể.

Mục tiêu đề tài

- Trình bày các khái niệm cơ bản về cơ sở dữ liệu phân tán và bài toán thiết kế phân tán.
- Tìm hiểu các kiểu phân mảnh ngang, dọc và các quy tắc đúng đắn của phân mảnh.
- Tìm hiểu sâu về phân mảnh hỗn hợp: định nghĩa, hai chiến lược HV và VH, cây phân mảnh, cách tái thiết và kiểm tra tính đúng đắn.
- Thiết kế phân mảnh hỗn hợp và cấp phát cho hệ thống quản lý đơn hàng, sau đó cài đặt thực nghiệm trên SQL Server.

CHƯƠNG 1. TỔNG QUAN VỀ CƠ SỞ DỮ LIỆU PHÂN TÁN

1.1 Khái niệm cơ sở dữ liệu phân tán

Theo Özsu và Valduriez [1], **cơ sở dữ liệu phân tán** (CSDLPT) là một tập hợp nhiều cơ sở dữ liệu có liên quan với nhau về mặt logic, được phân bố trên các nút của một mạng máy tính. **Hệ quản trị cơ sở dữ liệu phân tán** (DDBMS) là phần mềm quản lý CSDLPT, đồng thời làm cho sự phân tán trở nên **trong suốt** đối với người dùng.

Từ định nghĩa trên, có thể rút ra hai đặc điểm quan trọng:

- **Phân tán về vật lý:** dữ liệu không nằm trên một máy mà được lưu tại nhiều *site*. Mỗi site có hệ quản trị CSDL cục bộ, có khả năng xử lý độc lập, và các site trao đổi dữ liệu qua mạng.
- **Thống nhất về logic:** dữ liệu ở các site không rời rạc mà thuộc về cùng một lược đồ toàn cục. Một tập hợp file đặt rải rác trên nhiều máy, hay nhiều CSDL độc lập không liên quan đến nhau, đều không phải là CSDLPT.

1.2 So sánh CSDL tập trung và CSDL phân tán

Trong mô hình tập trung, toàn bộ dữ liệu đặt tại một máy chủ, các máy khách ở xa truy cập qua mạng. Mô hình này đơn giản nhưng có những hạn chế rõ ràng khi tổ chức trải rộng về địa lý. Bảng 1.1 so sánh hai mô hình theo các tiêu chí chính.

Bảng 1.1: *So sánh CSDL tập trung và CSDL phân tán*

Tiêu chí	CSDL tập trung	CSDL phân tán
Vị trí dữ liệu	Một máy chủ duy nhất	Nhiều site nối mạng
Truy cập từ chi nhánh	Luôn phải truy cập từ xa	Phần lớn truy cập cục bộ
Độ sẵn sàng	Máy chủ hỏng thì cả hệ thống dừng	Một site hỏng, các site khác vẫn hoạt động
Khả năng mở rộng	Nâng cấp máy chủ (mở rộng theo chiều dọc)	Thêm site mới (mở rộng theo chiều ngang)
Độ phức tạp	Thấp	Cao: thiết kế phân tán, truy vấn và giao dịch phân tán
Kiểm soát và bảo mật	Tập trung, dễ quản lý	Phải bảo vệ dữ liệu ở nhiều nơi và trên đường truyền

1.3 Kiến trúc tham chiếu của CSDL phân tán

Kiến trúc tham chiếu của CSDLPT [1], [2] mô tả dữ liệu qua nhiều mức lược đồ:

- **Lược đồ toàn cục:** định nghĩa toàn bộ dữ liệu của hệ thống dưới dạng các quan hệ, giống như một CSDL tập trung. Người dùng và ứng dụng làm việc ở mức này.
- **Lược đồ phân mảnh:** mô tả cách mỗi quan hệ toàn cục được chia thành các mảnh, bằng các phép chọn (σ) và phép chiếu (π) của đại số quan hệ.
- **Lược đồ cấp phát:** xác định mỗi mảnh được đặt ở site nào. Một mảnh có thể được đặt ở một site (không nhân bản) hoặc ở nhiều site (nhân bản).
- **Lược đồ ánh xạ cục bộ:** ánh xạ các mảnh vật lý tại mỗi site sang các đối tượng của DBMS cục bộ, chẳng hạn bảng của SQL Server hoặc Oracle.

1.4 Tính trong suốt trong CSDL phân tán

Tính trong suốt nghĩa là người dùng không cần biết các chi tiết cài đặt của hệ thống phân tán. Có thể phân biệt các mức trong suốt sau, theo thứ tự từ cao xuống thấp:

- **Trong suốt phân mảnh:** người dùng truy vấn trên quan hệ toàn cục mà không cần biết quan hệ đó đã bị chia thành những mảnh nào. Đây là mức trong suốt cao nhất.
- **Trong suốt vị trí:** người dùng biết các mảnh nhưng không cần biết mỗi mảnh nằm ở site nào.
- **Trong suốt nhân bản:** người dùng không cần biết một mảnh có bao nhiêu bản sao, cũng không phải tự cập nhật từng bản sao.
- **Trong suốt ánh xạ cục bộ:** người dùng không cần biết mỗi site dùng hệ quản trị nào và lưu dữ liệu ra sao.

Mức trong suốt càng cao thì việc viết ứng dụng càng đơn giản, nhưng DDBMS phải đảm nhận nhiều việc hơn, ví dụ phải tự phân rã một truy vấn toàn cục thành các truy vấn trên từng mảnh. Trong thực nghiệm ở Chương 4, nhóm đạt được trong suốt phân mảnh và trong suốt vị trí bằng view và synonym của SQL Server.

1.5 Thiết kế cơ sở dữ liệu phân tán

Trong thiết kế từ trên xuống, sau khi có lược đồ toàn cục, người thiết kế phải giải quyết hai bài toán:

- **Phân mảnh:** chia mỗi quan hệ thành các mảnh sao cho mỗi mảnh chứa đúng phần dữ liệu mà một nhóm ứng dụng cần. Đây là nội dung chính của Chương 2 và Chương 3.
- **Cấp phát:** đặt các mảnh lên các site sao cho tối thiểu hóa chi phí truy cập và truyền thông, đồng thời đáp ứng yêu cầu về độ sẵn sàng. Có ba phương án: không nhân bản (mỗi mảnh ở đúng một site), nhân bản một phần, và nhân bản toàn phần.

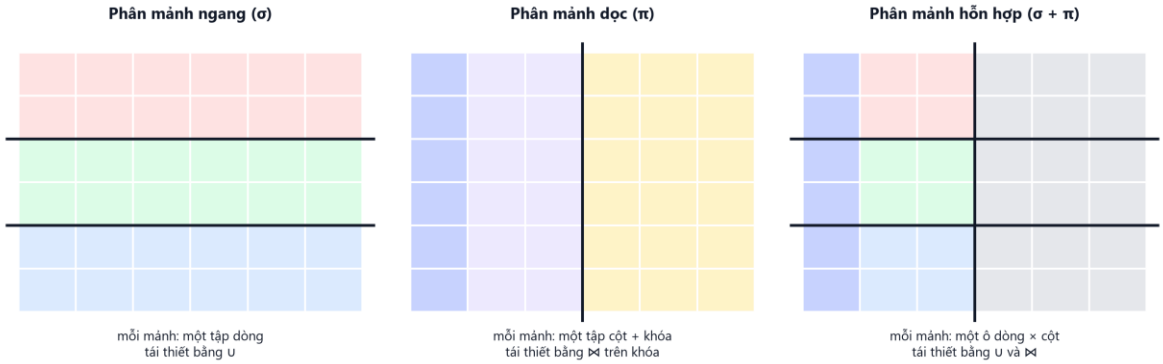
CHƯƠNG 2. PHÂN MẢNH DỮ LIỆU TRONG CSDL PHÂN TÁN

2.1 Lý do và đơn vị phân mảnh

Nếu đơn vị phân tán là cả một quan hệ, sẽ có hai lựa chọn đều không hiệu quả. Đặt quan hệ ở một site thì các site khác phải truy cập từ xa. Nhân bản quan hệ ra mọi site thì tốn không gian lưu trữ, và mỗi lần cập nhật phải sửa tất cả các bản sao. Trong khi đó, mỗi ứng dụng thường chỉ dùng một **tập con** của quan hệ, ví dụ chi nhánh Hà Nội chỉ cần khách hàng ở Hà Nội. Vì vậy, đơn vị phân tán hợp lý là các **mảnh** (fragment) của quan hệ.

Phân mảnh cũng có cái giá phải trả. Những ứng dụng cần dữ liệu ở nhiều mảnh phải ghép các mảnh lại, phát sinh chi phí truyền thông và xử lý. Việc kiểm tra ràng buộc toàn vẹn liên quan đến nhiều mảnh cũng phức tạp hơn. Do đó **mức độ phân mảnh** phải vừa phải: không phân mảnh thì mất tính cục bộ, phân mảnh quá nhỏ (đến từng bộ hoặc từng thuộc tính) thì chi phí ghép lại rất lớn. Mức độ hợp lý được xác định dựa trên các ứng dụng sử dụng CSDL.

Có ba kiểu phân mảnh cơ bản (Hình 2.1): **phân mảnh ngang** (theo dòng), **phân mảnh dọc** (theo cột) và **phân mảnh hỗn hợp** (kết hợp cả hai).



Hình 2.1: *Minh họa ba kiểu phân mảnh*

2.2 Các quy tắc đúng đắn của phân mảnh

Giả sử quan hệ R được phân mảnh thành các mảnh R_1, R_2, …, R_n. Phân mảnh được gọi là đúng đắn nếu thỏa ba quy tắc [1]:

- **Tính đầy đủ (completeness):** mỗi mục dữ liệu của R (một bộ đối với phân mảnh ngang, một thuộc tính đối với phân mảnh dọc) đều có mặt trong ít nhất một mảnh R_i. Quy tắc này đảm bảo phân mảnh không làm mất dữ liệu.
- **Tính tái thiết (reconstruction):** tồn tại một phép toán quan hệ ∇ sao cho $R = \nabla R_i$. Với phân mảnh ngang, ∇ là phép hợp (∪); với phân mảnh dọc, ∇ là phép kết

nối (⋈) trên khóa. Quy tắc này đảm bảo các ràng buộc phụ thuộc giữa dữ liệu được bảo toàn.

- **Tính rời nhau (disjointness):** với phân mảnh ngang, một bộ thuộc R_j thì không thuộc R_k ($j \neq k$). Với phân mảnh dọc, các mảnh chỉ được trùng nhau ở khóa chính; khóa phải được lặp lại ở mọi mảnh để có thể tái thiết.

2.3 Phân mảnh ngang nguyên thủy

Phân mảnh ngang chia quan hệ theo các bộ. Mỗi mảnh là kết quả của một phép chọn trên quan hệ:

$$R_i = \sigma_{F_i}(R), \quad i = 1..n$$

trong đó F_i là vị từ chọn của mảnh R_i. Gọi là phân mảnh **nguyên thủy** vì vị từ được định nghĩa trên chính các thuộc tính của R.

- **Vị từ đơn giản** có dạng p_j: A_i θ Giá_trị, với A_i là thuộc tính của R và $\theta \in \{=, <, \leq, >, \geq, \neq\}$. Ví dụ: p_1: MaCN = 'CN01'.
- **Vị từ hội sơ cấp (minterm)** trên tập vị từ đơn giản $Pr = \{p_1, \ldots, p_m\}$ là phép hội của tất cả các vị từ trong Pr, mỗi vị từ xuất hiện ở dạng khẳng định hoặc phủ định.

Mỗi minterm xác định một mảnh ngang, gọi là **mảnh hội sơ cấp**. Với m vị từ sẽ có 2^m minterm, nhưng nhiều minterm mâu thuẫn và bị loại bỏ dựa trên ngữ nghĩa của thuộc tính.

Ví dụ. Xét quan hệ KhachHang(MaKH, HoTen, SDT, DiaChi, MaCN, HanMucNo, CongNo), với MaCN ∈ {'CN01', 'CN02', 'CN03'}. Ứng dụng tra cứu khách hàng ở mỗi chi nhánh chỉ truy cập khách thuộc chi nhánh đó, nên tập vị từ ứng viên là:

$$p_1: \text{MaCN} = \text{'CN01'} \quad p_2: \text{MaCN} = \text{'CN02'} \quad p_3: \text{MaCN} = \text{'CN03'}$$

Có $2^3 = 8$ minterm. Vì MaCN chỉ nhận một giá trị nên có các phép kéo theo $p_1 \Rightarrow \neg p_2 \wedge \neg p_3$ (tương tự cho p_2, p_3). Mọi minterm chứa từ hai vị từ khẳng định trở lên đều mâu thuẫn. Minterm $\neg p_1 \wedge \neg p_2 \wedge \neg p_3$ cũng mâu thuẫn vì miền giá trị của MaCN chỉ gồm ba giá trị trên. Còn lại ba minterm, rút gọn được thành p_1, p_2, p_3, cho ba mảnh:

$$KH_i = \sigma_{\text{MaCN='CN0}i\text{'}}(\text{KhachHang}), \quad i = 1, 2, 3$$

Ba mảnh KH_1, KH_2, KH_3 thỏa các quy tắc đúng đắn: **đầy đủ** vì $p_1 \vee p_2 \vee p_3$ luôn đúng trên miền giá trị của MaCN; **rời nhau** vì các vị từ loại trừ lẫn nhau; **tái thiết** được bằng KhachHang = $KH_1 \cup KH_2 \cup KH_3$.

2.4 Phân mảnh ngang dẫn xuất

Phân mảnh ngang dẫn xuất áp dụng cho một quan hệ **thành viên** (member), dựa trên phân mảnh của quan hệ **chủ** (owner) mà nó tham chiếu tới. Quan hệ chủ và thành viên được nối với nhau bằng một liên kết, thường là khóa ngoại. Nếu S là quan hệ chủ đã được phân mảnh thành S_1, …, S_n, thì quan hệ thành viên R được phân mảnh bằng phép nửa kết nối:

$$R_i = R \ltimes S_i, \quad i = 1..n$$

Quan hệ DonHang(MaDH, NgayDat, MaKH, MaNV, TrangThai, TongTien) tham chiếu KhachHang qua MaKH. Để đơn hàng nằm cùng site với khách hàng, DonHang được phân mảnh dẫn xuất theo các mảnh KH_i ở mục 2.3:

$$DH_i = \text{DonHang} \ltimes_{\text{MaKH}} KH_i, \quad i = 1, 2, 3$$

Tương tự, ChiTietDonHang được phân mảnh dẫn xuất theo DonHang: $CTDH_i$ = ChiTietDonHang $\ltimes_{\text{MaDH}} DH_i$.

Lợi ích quan trọng nhất: các mảnh KH_i, DH_i, $CTDH_i$ cùng đặt tại một site, nên phép kết nối KhachHang ⋈ DonHang ⋈ ChiTietDonHang được thực hiện **cục bộ** tại từng site rồi hợp kết quả lại, không cần truyền dữ liệu giữa các site. Nhờ vậy đồ thị kết nối giữa các mảnh trở nên **đơn giản**: mỗi mảnh của quan hệ thành viên chỉ kết nối với đúng một mảnh của quan hệ chủ.

- **Đầy đủ:** cần ràng buộc toàn vẹn tham chiếu. Nếu một đơn hàng có MaKH không tồn tại trong KhachHang, đơn đó sẽ không thuộc mảnh nào.
- **Rời nhau:** được đảm bảo khi mỗi bộ của quan hệ thành viên chỉ kết nối với bộ của đúng một mảnh chủ. Ở đây mỗi đơn hàng có đúng một MaKH, và mỗi khách hàng thuộc đúng một chi nhánh.
- **Tái thiết:** DonHang = $DH_1 \cup DH_2 \cup DH_3$.

2.5 Phân mảnh dọc

Phân mảnh dọc chia quan hệ theo các thuộc tính. Mỗi mảnh là phép chiếu của R lên một tập thuộc tính, và **mọi mảnh đều chứa khóa chính** K của R:

$$R_i = \pi_{K \cup A_i}(R), \quad i = 1..n$$

Tái thiết bằng phép kết nối trên khóa: $R = R_1 \bowtie_K R_2 \bowtie_K \ldots \bowtie_K R_n$. Mục tiêu của phân mảnh dọc là gom vào cùng một mảnh những thuộc tính thường được truy cập cùng nhau, để mỗi ứng dụng chỉ cần đọc một mảnh.

Bài toán phân mảnh dọc phức tạp hơn phân mảnh ngang: với m thuộc tính không khóa, số cách chia là số Bell B(m), tăng rất nhanh (B(10) ≈ 115.000). Vì vậy người ta

dùng các phương pháp heuristic, phổ biến nhất là thuật toán năng lượng liên kết BEA [4], [5] kết hợp với thuật toán SPLIT.

Ví dụ. Xét các thuộc tính không khóa của KhachHang: A_1 = HoTen, A_2 = SDT, A_3 = DiaChi, A_4 = HanMucNo, A_5 = CongNo (khóa MaKH được lặp lại ở mọi mảnh nên không tham gia gom cụm). Có bốn ứng dụng: q_1 tra cứu khách hàng, q_2 in phiếu giao hàng, q_3 kiểm tra hạn mức, q_4 báo cáo công nợ. Ma trận sử dụng cho ở Bảng 2.1.

Bảng 2.1: *Ma trận sử dụng thuộc tính use(q_i, A_j)*

	HoTen	**SDT**	**DiaChi**	**HanMucNo**	**CongNo**
q_1	1	1	1	0	0
q_2	1	0	1	0	0
q_3	0	0	0	1	1
q_4	0	0	0	0	1

Áp dụng BEA và SPLIT, điểm cắt tốt nhất nằm sau HanMucNo. Kết quả là hai mảnh dọc, mỗi mảnh đều chứa khóa MaKH:

$$\text{KH_TC} = \pi_{\text{MaKH, HanMucNo, CongNo}}(\text{KhachHang}), \quad \text{KH_GD} = \pi_{\text{MaKH, HoTen, SDT, DiaChi}}(\text{KhachHang})$$

Mảnh KH_GD phục vụ các ứng dụng tại chi nhánh; mảnh KH_TC phục vụ phòng kế toán tại trụ sở. Không có ứng dụng nào phải dùng cả hai mảnh.

2.6 Cấp phát và nhân bản

Sau khi phân mảnh, mỗi mảnh cần được đặt lên một hoặc nhiều site. Bài toán cấp phát tối ưu là bài toán NP-khó, nên trong thực tế thường dùng các nguyên tắc heuristic:

- Đặt mảnh tại site truy cập nó nhiều nhất.
- Mảnh chủ yếu được đọc và ít được cập nhật thì nên **nhân bản** ra nhiều site để tăng tính cục bộ và độ sẵn sàng.
- Mảnh được cập nhật thường xuyên thì nên giữ **một bản duy nhất**, tránh chi phí đồng bộ giữa các bản sao.
- Các mảnh thường được kết nối với nhau (như các mảnh dẫn xuất) nên đặt cùng site.

CHƯƠNG 3. PHÂN MẢNH HỖN HỢP

3.1 Định nghĩa và động cơ sử dụng

Phân mảnh ngang phù hợp khi các ứng dụng khác nhau ở **những bộ** mà chúng truy cập. Phân mảnh dọc phù hợp khi các ứng dụng khác nhau ở **những thuộc tính** mà chúng truy cập. Trong nhiều hệ thống thực tế, hai sự khác biệt này xuất hiện đồng thời. Bảng 3.1 mô tả nhu cầu truy cập quan hệ KhachHang của các nhóm người dùng trong một công ty có ba chi nhánh.

Bảng 3.1: *Nhu cầu truy cập KhachHang theo dòng và theo cột*

Nhóm người dùng	Những bộ cần truy cập	Những thuộc tính cần truy cập
Nhân viên chi nhánh Hà Nội	Khách của chi nhánh Hà Nội	Nhóm giao dịch: HoTen, SDT, Email, DiaChi
Nhân viên chi nhánh Đà Nẵng	Khách của chi nhánh Đà Nẵng	Nhóm giao dịch
Nhân viên chi nhánh TP.HCM	Khách của chi nhánh TP.HCM	Nhóm giao dịch
Phòng kế toán (trụ sở)	Khách của cả ba chi nhánh, báo cáo theo từng miền	Nhóm tài chính: HanMucNo, CongNo, DiemTichLuy

Nếu chỉ phân mảnh ngang theo chi nhánh, mỗi chi nhánh vẫn phải lưu các cột tài chính mà mình không dùng, và phòng kế toán phải truy cập cả ba chi nhánh để lấy công nợ. Nếu chỉ phân mảnh dọc, mỗi chi nhánh phải lưu khách hàng của các chi nhánh khác. Không kiểu phân mảnh đơn lẻ nào đáp ứng được cả hai nhu cầu.

Phân mảnh hỗn hợp (hybrid fragmentation, còn gọi là phân mảnh lai) là việc áp dụng **nối tiếp** phân mảnh ngang và phân mảnh dọc lên một quan hệ: kết quả của một bước phân mảnh lại được phân mảnh tiếp theo kiểu còn lại [1], [2]. Một mảnh hỗn hợp vì vậy được biểu diễn bằng biểu thức kết hợp phép chọn và phép chiếu, có hai dạng:

$$R_{ij} = \pi_{K \cup A_j}(\sigma_{F_i}(R)) \qquad \text{hoặc} \qquad R_{jk} = \sigma_{F_k}(\pi_{K \cup A_j}(R))$$

trong đó K là khóa chính của R, A_j là một nhóm thuộc tính, F_i và F_k là các vị từ chọn.

3.2 Chiến lược ngang trước, dọc sau (HV)

Trong chiến lược HV, quan hệ được phân mảnh ngang trước, sau đó **từng** mảnh ngang được phân mảnh dọc:

1. Phân mảnh ngang nguyên thủy R thành $R_1, \ldots, R_n$ với $R_i = \sigma_{F_i}(R)$, như ở mục 2.3.

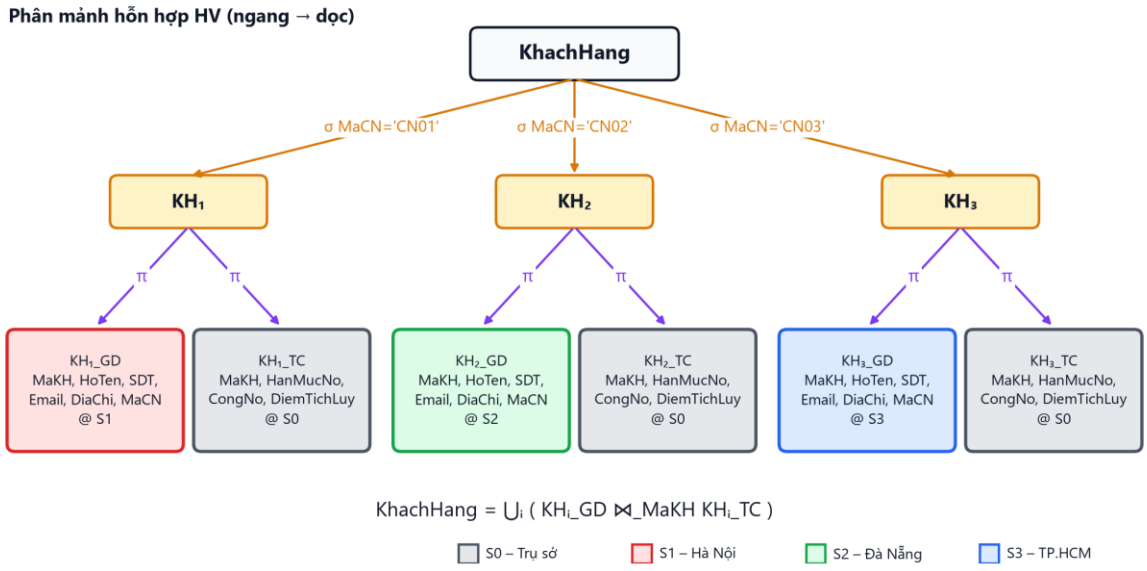
2. Với mỗi R_i, xác định các nhóm thuộc tính bằng ma trận ái lực, BEA và SPLIT (mục 2.5), tính trên các ứng dụng truy cập R_i, rồi chia R_i thành các mảnh $R_{ij} = \pi_{K \cup A_{ij}}(R_i)$.

Đặc điểm của HV:

– Mỗi mảnh ngang có thể được chia dọc theo **cách khác nhau**, vì các ứng dụng truy cập mỗi mảnh ngang có thể khác nhau.

– Phù hợp khi tiêu chí phân biệt **chủ yếu là theo bộ** (theo địa lý, theo đơn vị quản lý), còn sự khác biệt theo thuộc tính là thứ yếu và có thể phụ thuộc vào từng nhóm bộ.

Hình 3.1 là cây phân mảnh HV của quan hệ KhachHang trong bài toán ở Chương 4: ba mảnh ngang theo chi nhánh, mỗi mảnh tách thành phần giao dịch (GD) và phần tài chính (TC), tổng cộng sáu mảnh lá.



Hình 3.1: *Cây phân mảnh hỗn hợp HV của quan hệ KhachHang*

3.3 Chiến lược dọc trước, ngang sau (VH)

Trong chiến lược VH, quan hệ được phân mảnh dọc trước, sau đó **một số** mảnh dọc được phân mảnh ngang tiếp:

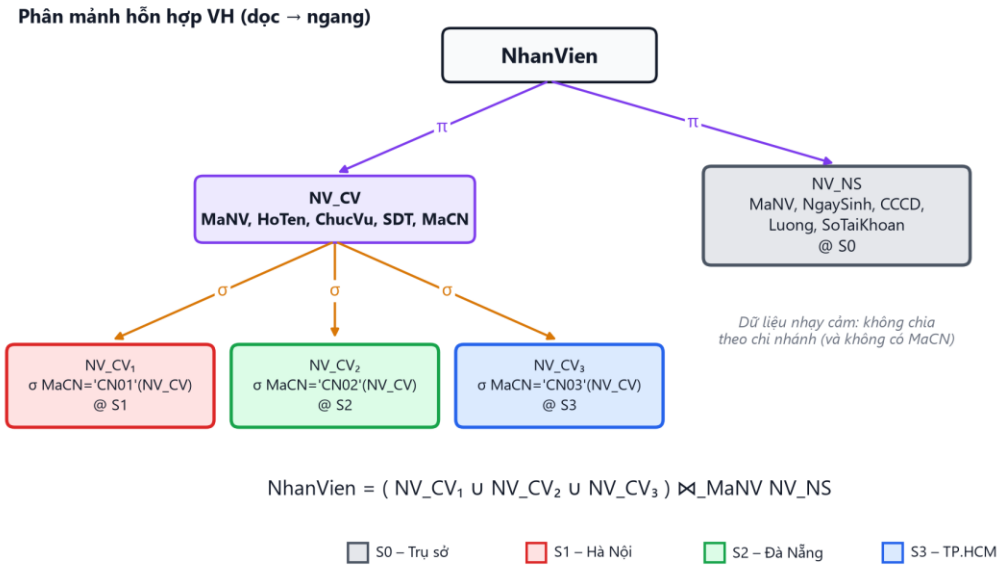
1. Phân mảnh dọc R thành các mảnh $R_j = \pi_{K \cup A_j}(R)$ bằng BEA và SPLIT, tính trên toàn bộ ứng dụng.
2. Với những mảnh dọc mà các ứng dụng sử dụng chúng chỉ truy cập một phần các bộ, phân mảnh ngang tiếp: $R_{jk} = \sigma_{F_{jk}}(R_j)$.

Đặc điểm của VH:

– Chỉ những mảnh dọc **cần thiết** mới bị chia ngang; các mảnh dọc khác được giữ nguyên. Nhờ vậy số mảnh ít hơn và việc quản lý đơn giản hơn.

– Phù hợp khi tiêu chí phân biệt **chủ yếu là theo thuộc tính**, chẳng hạn tách dữ liệu nhạy cảm để bảo mật. Phần nhạy cảm thường được xử lý tập trung nên không cần chia ngang.

Hình 3.2 là cây phân mảnh VH của quan hệ NhanVien. Thông tin nhân sự NV_NS (ngày sinh, CCCD, lương, số tài khoản) được tách ra và giữ tập trung tại trụ sở. Chỉ phần thông tin công việc NV_CV được chia tiếp theo chi nhánh.



Hình 3.2: *Cây phân mảnh hỗn hợp VH của quan hệ NhanVien*

Phép chọn và phép chiếu giao hoán với nhau khi các thuộc tính của vị từ nằm trong tập thuộc tính được chiếu. Vì vậy, nếu **mọi** mảnh dọc đều chứa thuộc tính của vị từ và đều được chia ngang theo **cùng** các vị từ, HV và VH cho cùng một tập mảnh lá. Hai chiến lược chỉ thực sự khác nhau khi cây phân mảnh **không cân đối**: chỉ một số nút được chia tiếp (VH), hoặc mỗi nút được chia theo cách khác nhau (HV).

3.4 Cây phân mảnh và tái thiết quan hệ toàn cục

Cây phân mảnh là cách biểu diễn trực quan một lược đồ phân mảnh hỗn hợp:

– **Gốc** là quan hệ toàn cục R.

– Mỗi **nút trong** là một mảnh trung gian. Các cạnh từ một nút trong xuống các con của nó đều cùng một kiểu phân mảnh: hoặc đều là phép chọn σ (chia ngang), hoặc đều là phép chiếu π (chia dọc).

– Mỗi **lá** là một mảnh cuối cùng, được lưu trữ vật lý và cấp phát cho site.

Quan hệ toàn cục được tái thiết bằng cách duyệt cây phân mảnh **từ lá lên gốc**. Tại mỗi nút trong:

– nếu các con được tạo bằng phép chọn σ: nút = **hợp (∪)** các con;

– nếu các con được tạo bằng phép chiếu π: nút = **kết nối (⋈) trên khóa K** các con.

Với hai ví dụ ở trên:

$$\text{KhachHang} = \bigcup_{i=1..3} (\text{KH}_i\text{_GD} \bowtie_{\text{MaKH}} \text{KH}_i\text{_TC})$$

$$\text{NhanVien} = (\text{NV_CV}_1 \cup \text{NV_CV}_2 \cup \text{NV_CV}_3) \bowtie_{\text{MaNV}} \text{NV_NS}$$

Trong cài đặt bằng SQL, phép hợp các mảnh ngang rời nhau có thể dùng UNION ALL thay cho UNION. Vì các mảnh đã rời nhau, không có bộ trùng cần loại bỏ, nên UNION ALL cho cùng kết quả mà không tốn chi phí sắp xếp hay băm để khử trùng lặp.

3.5 Kiểm tra tính đúng đắn

Mệnh đề. Nếu phân mảnh tại mọi nút trong của cây phân mảnh đều thỏa ba quy tắc đầy đủ, tái thiết và rời nhau, thì phân mảnh hỗn hợp tương ứng với toàn bộ cây cũng thỏa ba quy tắc đó.

Chứng minh (phác thảo) bằng quy nạp theo chiều cao của cây:

– **Đầy đủ:** mỗi mục dữ liệu của nút cha có mặt trong ít nhất một nút con (giả thiết tại nút cha). Theo giả thiết quy nạp, mỗi mục dữ liệu của nút con có mặt trong ít nhất một lá của cây con. Vậy mọi mục dữ liệu của R đều có mặt trong ít nhất một lá.

– **Tái thiết:** mỗi nút con tái thiết được từ các lá của nó (giả thiết quy nạp), và nút cha tái thiết được từ các con bằng ∪ hoặc ⋈ (giả thiết tại nút cha). Hợp thành các phép toán đó cho biểu thức tái thiết R từ các lá, như ở mục 3.4.

– **Rời nhau:** hai lá bất kỳ có một nút tổ tiên chung gần nhất, và chúng nằm trong hai cây con của hai nút con khác nhau. Nếu nút tổ tiên đó chia ngang, hai lá chứa các tập bộ rời nhau. Nếu nút đó chia dọc, hai lá chỉ có chung thuộc tính khóa K. Trường hợp nào thì quy tắc rời nhau cũng thỏa.

3.6 Ảnh hưởng đến xử lý truy vấn và giao dịch

Khi nhận một truy vấn trên quan hệ toàn cục, DDBMS thay quan hệ đó bằng biểu thức tái thiết (gọi là **chương trình định vị**, localization program), rồi rút gọn [1]. Ví dụ với quan hệ KhachHang:

– Truy vấn SELECT HoTen, SDT FROM KhachHang WHERE MaCN = 'CN01' chỉ cần mảnh KH_1_GD, chạy hoàn toàn tại chi nhánh Hà Nội.

– Truy vấn SELECT SUM(CongNo) FROM KhachHang WHERE MaCN = 'CN02' chỉ cần mảnh KH_2_TC, chạy hoàn toàn tại trụ sở.

– Truy vấn SELECT HoTen, CongNo FROM KhachHang WHERE MaCN = 'CN02' cần KH_2_GD ⋈ KH_2_TC: hai mảnh nằm ở hai site khác nhau, phải truyền dữ liệu qua mạng.

Một thao tác cập nhật chạm đến thuộc tính thuộc nhiều mảnh dọc (ví dụ lập đơn hàng ở chi nhánh và cộng công nợ ở trụ sở) trở thành **giao dịch phân tán**, cần giao thức cam kết hai pha (2PC) để đảm bảo tính nguyên tử.

3.7 Ưu điểm, hạn chế và so sánh

Bảng 3.2 tóm tắt so sánh ba kiểu phân mảnh.

Bảng 3.2: *So sánh ba kiểu phân mảnh*

Tiêu chí	Phân mảnh ngang	Phân mảnh dọc	Phân mảnh hỗn hợp
Đơn vị chia	Tập bộ	Tập thuộc tính + khóa	Tập bộ × tập thuộc tính
Phép toán	σ (hoặc ⋉ nếu dẫn xuất)	π	Kết hợp σ và π
Tái thiết	∪	⋈ trên khóa	∪ và ⋈ theo cây phân mảnh
Phù hợp khi	Ứng dụng khác nhau theo bộ (địa lý, đơn vị)	Ứng dụng khác nhau theo thuộc tính, cần bảo mật cột	Ứng dụng khác nhau theo cả bộ lẫn thuộc tính
Độ phức tạp	Thấp	Trung bình	Cao

Để cài đặt phân mảnh hỗn hợp, cách phổ biến là: dùng cơ chế chia ngang (bảng riêng có CHECK, hoặc partitioning) cho bước ngang; dùng các bảng riêng chứa khóa chung cho bước dọc; và dùng view kết hợp UNION ALL và JOIN để tái thiết [3], [6]. Đây là cách nhóm áp dụng ở Chương 4.

CHƯƠNG 4. ỨNG DỤNG PHÂN MẢNH HỖN HỢP CHO HỆ THỐNG QUẢN LÝ ĐƠN HÀNG

4.1 Mô tả bài toán

Công ty thương mại X (công ty giả định) phân phối thiết bị văn phòng qua ba chi nhánh ở ba miền và có một trụ sở chính đặt tại TP. Hồ Chí Minh. Mỗi địa điểm là một site của CSDL phân tán.

Các nghiệp vụ chính:

- Mỗi khách hàng được quản lý bởi **đúng một** chi nhánh, là nơi khách đăng ký.
- Nhân viên chi nhánh tra cứu khách hàng, lập đơn hàng và cập nhật trạng thái đơn cho khách của chi nhánh mình. Đây là nghiệp vụ diễn ra nhiều nhất.
- Phòng Kế toán quản lý hạn mức công nợ, công nợ hiện tại và điểm tích lũy của khách hàng ở cả ba miền, lập báo cáo công nợ theo từng miền.
- Phòng Nhân sự quản lý ngày sinh, số CCCD, lương và số tài khoản của nhân viên. Đây là dữ liệu nhạy cảm, chi nhánh không được truy cập.
- Danh mục sản phẩm và giá bán do trụ sở quản lý; mọi chi nhánh đều cần đọc khi lập đơn.

Lược đồ toàn cục gồm sáu quan hệ:

- ChiNhanh(**MaCN**, TenCN, KhuVuc, DiaChi, SDT)
- KhachHang(**MaKH**, HoTen, SDT, Email, DiaChi, MaCN, HanMucNo, CongNo, DiemTichLuy)
- NhanVien(**MaNV**, HoTen, ChucVu, SDT, MaCN, NgaySinh, CCCD, Luong, SoTaiKhoan)
- SanPham(**MaSP**, TenSP, DonViTinh, GiaBan, MoTa)
- DonHang(**MaDH**, NgayDat, MaKH, MaNV, TrangThai, TongTien)
- ChiTietDonHang(**MaDH, MaSP**, SoLuong, DonGia)

Ràng buộc miền giá trị MaCN ∈ {'CN01', 'CN02', 'CN03'} được khai báo bằng CHECK. Ràng buộc này là cơ sở để chứng minh tính đầy đủ của phân mảnh ngang.

4.2 Phân tích ứng dụng

Theo phương pháp thiết kế từ trên xuống, nhóm liệt kê các ứng dụng chính, nơi phát sinh, dữ liệu sử dụng và tần suất (Bảng 4.1).

Bảng 4.1: *Các ứng dụng chính của hệ thống*

Mã	Ứng dụng	Phát sinh tại	Dữ liệu sử dụng	Tần suất
Q1	Tra cứu khách hàng của chi nhánh để lập đơn	S1, S2, S3	KhachHang(MaKH, HoTen, SDT, Email, DiaChi, MaCN)	Rất cao
Q2	Lập đơn, xem và cập nhật đơn hàng của chi nhánh	S1, S2, S3	DonHang, ChiTietDonHang, SanPham	Rất cao
Q3	Kiểm tra hạn mức, công nợ; báo cáo công nợ theo miền	S0	KhachHang(MaKH, MaCN, HanMucNo, CongNo, DiemTichLuy)	Trung bình
Q4	Xem danh sách nhân viên chi nhánh	S1, S2, S3	NhanVien(MaNV, HoTen, ChucVu, SDT, MaCN)	Trung bình
Q5	Tính lương, quản lý hồ sơ nhân sự	S0	NhanVien(MaNV, NgaySinh, CCCD, Luong, SoTaiKhoan)	Thấp
Q6	Báo cáo doanh thu toàn công ty, theo chi nhánh	S0	DonHang, ChiTietDonHang	Thấp
Q7	Cập nhật danh mục và giá sản phẩm	S0	SanPham	Thấp

Ma trận sử dụng thuộc tính của KhachHang cho thấy các thuộc tính chia thành hai nhóm tách biệt: **nhóm giao dịch** do Q1, Q2 sử dụng tại chi nhánh và **nhóm tài chính** do Q3 sử dụng tại trụ sở. Hai nhóm chỉ chung khóa MaKH và thuộc tính MaCN. Kết quả này phù hợp với ví dụ BEA và SPLIT ở mục 2.5. Đồng thời, mỗi chi nhánh chỉ truy cập các bộ của khách thuộc chi nhánh mình. Như vậy KhachHang cần được phân mảnh theo cả bộ lẫn thuộc tính. Lập luận tương tự áp dụng cho NhanVien: Q4 dùng thông tin công việc tại chi nhánh, Q5 dùng thông tin nhân sự tại trụ sở.

4.3 Thiết kế phân mảnh

Bảng 4.2 tóm tắt kiểu phân mảnh được chọn cho từng quan hệ.

Bảng 4.2: *Kiểu phân mảnh của từng quan hệ*

Quan hệ	Kiểu phân mảnh	Số mảnh lá	Lý do
KhachHang	Hỗn hợp HV	6	Chi nhánh cần dòng của mình + cột giao dịch; kế toán cần cột tài chính, báo cáo theo miền
NhanVien	Hỗn hợp VH	4	Tách dữ liệu nhân sự nhạy cảm, xử lý tập trung; chỉ chia ngang phần công việc
DonHang	Ngang dẫn xuất theo KhachHang	3	Đặt đơn cùng site với khách, kết nối cục bộ
ChiTietDonHang	Ngang dẫn xuất theo DonHang	3	Đặt chi tiết cùng site với đơn

Quan hệ	Kiểu phân mảnh	Số mảnh lá	Lý do
ChiNhanh, SanPham	Không phân mảnh, nhân bản toàn phần	—	Nhỏ, chủ yếu được đọc, ít cập nhật

KhachHang – phân mảnh hỗn hợp HV. Bước 1 – phân mảnh ngang nguyên thủy theo các vị từ p_i: MaCN = 'CN0i' (đã thiết kế ở mục 2.3). Bước 2 – phân mảnh dọc từng mảnh KH_i thành phần giao dịch và phần tài chính. Thuộc tính MaCN được giữ ở phần giao dịch vì đó là thuộc tính phân mảnh, cần cho ràng buộc CHECK tại chi nhánh:

$$KH_i_GD = \pi_{MaKH,\ HoTen,\ SDT,\ Email,\ DiaChi,\ MaCN}(KH_i) \quad \rightarrow \text{đặt tại } S_i$$

$$KH_i_TC = \pi_{MaKH,\ HanMucNo,\ CongNo,\ DiemTichLuy}(KH_i) \quad \rightarrow \text{đặt tại } S0$$

Nhóm chọn HV thay vì VH vì phòng Kế toán lập báo cáo công nợ **theo từng miền**. Phần tài chính được chia sẵn theo miền nên báo cáo của mỗi miền chỉ cần đọc đúng một mảnh.

NhanVien – phân mảnh hỗn hợp VH. Bước 1 – phân mảnh dọc thành thông tin công việc và thông tin nhân sự:

$$NV_CV = \pi_{MaNV,\ HoTen,\ ChucVu,\ SDT,\ MaCN}(NhanVien)$$

$$NV_NS = \pi_{MaNV,\ NgaySinh,\ CCCD,\ Luong,\ SoTaiKhoan}(NhanVien) \quad \rightarrow \text{đặt tại } S0$$

Bước 2 – phân mảnh ngang chỉ trên mảnh công việc:

$$NV_CV_i = \sigma_{MaCN='CN0i'}(NV_CV) \quad \rightarrow \text{đặt tại } S_i,\ \ i = 1, 2, 3$$

Nhóm chọn VH vì tiêu chí phân biệt chính ở đây là **thuộc tính**, nhằm tách dữ liệu nhạy cảm. Phần nhân sự được xử lý tập trung tại trụ sở nên không cần chia ngang; hơn nữa NV_NS không chứa MaCN nên cũng không chia ngang trực tiếp theo chi nhánh được.

DonHang, ChiTietDonHang – phân mảnh ngang dẫn xuất:

$$DH_i = DonHang \ltimes_{MaKH} KH_i, \quad CTDH_i = ChiTietDonHang \ltimes_{MaDH} DH_i, \quad i = 1, 2, 3$$

Nhờ phân mảnh dẫn xuất, mọi phép kết nối giữa khách hàng, đơn hàng và chi tiết đơn hàng đều thực hiện được tại chỗ ở chi nhánh.

ChiNhanh, SanPham – nhân bản toàn phần. Hai quan hệ này nhỏ (3 và 10 bộ trong dữ liệu mẫu), được đọc ở mọi site và chỉ trụ sở cập nhật. Chúng được nhân bản toàn phần ở cả bốn site.

4.4 Lược đồ cấp phát

Bảng 4.3: *Lược đồ cấp phát*

Site	Mảnh được cấp phát
S0 – Trụ sở	KH_1_TC, KH_2_TC, KH_3_TC, NV_NS, ChiNhanh*, SanPham*
S1 – Hà Nội	KH_1_GD, NV_CV_1, DH_1, $CTDH_1$, ChiNhanh*, SanPham*
S2 – Đà Nẵng	KH_2_GD, NV_CV_2, DH_2, $CTDH_2$, ChiNhanh*, SanPham*
S3 – TP.HCM	KH_3_GD, NV_CV_3, DH_3, $CTDH_3$, ChiNhanh*, SanPham*

(* bản nhân bản)

Với lược đồ cấp phát này, mức độ cục bộ của các ứng dụng như sau:

- **Q1, Q2, Q4** (tần suất cao) chạy **hoàn toàn cục bộ** tại chi nhánh: mọi mảnh cần thiết (KH_i_GD, DH_i, $CTDH_i$, NV_CV_i, SanPham) đều nằm tại S_i.
- **Q3, Q5, Q7** chạy cục bộ tại trụ sở.
- Chỉ **Q6** (báo cáo toàn công ty, tần suất thấp) cần lấy dữ liệu từ ba chi nhánh.

4.5 Cài đặt thực nghiệm trên SQL Server

- Hệ quản trị: Microsoft SQL Server 2022 Express (phiên bản 16.0.1000), công cụ SQL Server Management Studio 21. Toàn bộ kết quả trong mục này là kết quả chạy thật trên môi trường đó.
- Mỗi site được mô phỏng bằng một CSDL riêng trên cùng một instance: QLDH_TRUSO (S0), QLDH_HN (S1), QLDH_DN (S2), QLDH_HCM (S3). Thêm CSDL QLDH_GOC chứa lược đồ toàn cục và dữ liệu ban đầu, dùng làm chuẩn để đối chiếu.
- Dữ liệu mẫu: 3 chi nhánh, 30 khách hàng (10 khách mỗi chi nhánh), 12 nhân viên (4 người mỗi chi nhánh), 10 sản phẩm, 45 đơn hàng (15 đơn mỗi chi nhánh) với 97 dòng chi tiết.

Tại S0, nhóm tạo **synonym** [7] cho từng mảnh nằm ở chi nhánh, rồi xây dựng **view tái thiết** trên các synonym đó. Người dùng tại trụ sở truy vấn các view này như truy vấn trên quan hệ toàn cục, không cần biết dữ liệu được chia thành những mảnh nào và nằm ở site nào. Đó là **trong suốt phân mảnh** và **trong suốt vị trí**.

Để kiểm tra tính đúng đắn, nhóm viết 14 truy vấn so sánh dữ liệu trong các mảnh với CSDL gốc QLDH_GOC. Cả 14 phép kiểm tra đều cho số dòng lệch bằng 0, khẳng định phân mảnh đúng đắn trên dữ liệu thực (Hình 4.1).

```
SQLQuery1.sql...H\lixus (52))*
146         SELECT MaDH, MaSP, SoLuong, DonGia FROM QLDH_GOC.dbo.ChiTietDonHang) AS b)
147     + ABS((SELECT COUNT(*) FROM dbo.v_ChiTietDonHang) - (SELECT COUNT(*) FROM QLDH_GOC.dbo.ChiTietDonHang));
148
149 -- Kết quả (chụp màn hình bảng này cho slide 15 và mục 4.7)
150 SELECT STT, QuanHe, QuyTac, MoTa, SoDongLech,
151        CASE WHEN SoDongLech = 0 THEN N'ĐẠT' ELSE N'KHÔNG ĐẠT' END AS KetQua
152 FROM @KQ
153 ORDER BY STT;
154 -- Mong đợi: 14 dòng, tất cả SoDongLech = 0, KetQua = ĐẠT
155
100 %   No issues found
Results  Messages
```

	STT	QuanHe	QuyTac	MoTa	SoDongLech	KetQua
1	1	KhachHang	Đầy đủ	Bộ không thuộc mảnh ngang nào	0	ĐẠT
2	2	KhachHang	Đầy đủ	Thuộc tính không thuộc mảnh dọc nào (GD ∪ TC)	0	ĐẠT
3	3	KhachHang	Tái thiết	Bộ lệch giữa KhachHang gốc và v_KhachHang	0	ĐẠT
4	4	KhachHang	Rời nhau	MaKH thuộc nhiều hơn 1 mảnh ngang	0	ĐẠT
5	5	KhachHang	Rời nhau	Thuộc tính không khóa lặp ở cả GD và TC	0	ĐẠT
6	6	NhanVien	Đầy đủ	Thuộc tính không thuộc mảnh dọc nào (CV ∪ NS)	0	ĐẠT
7	7	NhanVien	Đầy đủ	Bộ của NV_CV không thuộc mảnh ngang nào	0	ĐẠT
8	8	NhanVien	Tái thiết	Bộ lệch giữa NhanVien gốc và v_NhanVien	0	ĐẠT
9	9	NhanVien	Rời nhau	MaNV thuộc nhiều hơn 1 mảnh ngang	0	ĐẠT
10	10	NhanVien	Rời nhau	Thuộc tính không khóa lặp ở cả CV và NS	0	ĐẠT
11	11	DonHang	Đầy đủ	Đơn hàng không thuộc mảnh nào	0	ĐẠT
12	12	DonHang	Tái thiết	Bộ lệch giữa DonHang gốc và v_DonHang	0	ĐẠT
13	13	DonHang	Rời nhau	MaDH thuộc nhiều hơn 1 mảnh	0	ĐẠT
14	14	ChiTietDonHang	Tái thiết	Bộ lệch giữa ChiTietDonHang gốc và v_ChiTietDonH...	0	ĐẠT

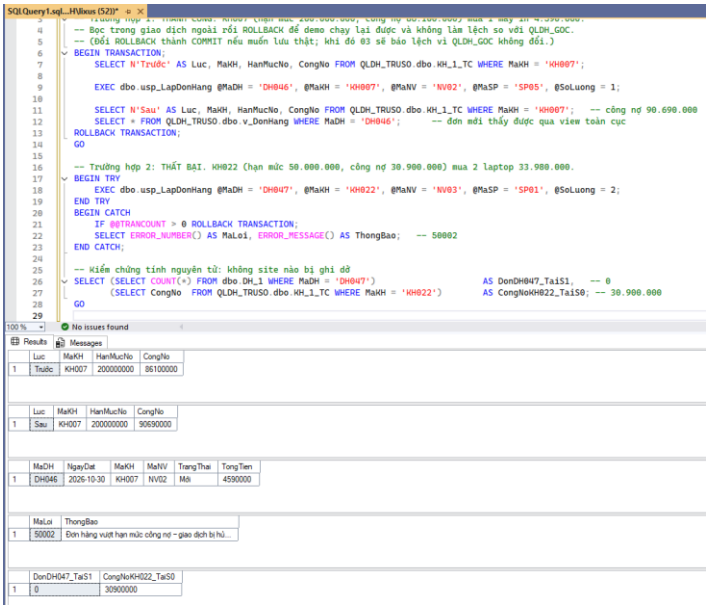
Hình 4.1: *Kết quả chạy phần kiểm tra trong 03_reconstruct_check.sql: 14/14 phép kiểm tra ĐẠT*

Truy vấn cục bộ tại chi nhánh (S1). Truy vấn Q2 liệt kê các đơn đang xử lý của chi nhánh Hà Nội, kèm tên khách, nhân viên và sản phẩm. Nó kết nối năm bảng nhưng tất cả đều nằm tại S1: DH_1, KH_1_GD, NV_CV_1, $CTDH_1$ và bản nhân bản SanPham. Phân mảnh dọc còn đóng vai trò bảo mật. Mảnh NV_CV_1 tại chi nhánh không có cột lương, nên câu lệnh SELECT MaNV, Luong FROM NV_CV_1 báo lỗi *Invalid column name 'Luong'*.

Truy vấn toàn cục qua view. Danh sách khách hàng có công nợ từ 80% hạn mức trở lên cần họ tên (ở chi nhánh) và công nợ (ở trụ sở), nên phải ghép mảnh của nhiều site. Truy vấn viết trên v_KhachHang như với một bảng bình thường. Kết quả gồm ba khách: KH011 (CN02, 90,1%), KH025 (CN01, 80,7%) và KH015 (CN03, 80,4%).

Giao dịch phân tán. Khi lập đơn hàng, chi nhánh ghi vào DH_i và $CTDH_i$ tại S_i, nhưng đồng thời phải kiểm tra hạn mức và cộng công nợ vào KH_i_TC tại S0. Một thao tác nghiệp vụ vì vậy ghi lên hai site và phải đảm bảo tính nguyên tử. Hai trường hợp được thử nghiệm (Hình 4.2):

- **Thành công:** khách KH007 (hạn mức 200.000.000 đồng, công nợ 86.100.000 đồng) mua một máy in giá 4.590.000 đồng. Đơn DH046 xuất hiện ở S1, đồng thời công nợ tại S0 tăng lên 90.690.000 đồng.
- **Thất bại:** khách KH022 (hạn mức 50.000.000 đồng, công nợ 30.900.000 đồng) mua hai laptop, tổng 33.980.000 đồng, vượt hạn mức. Thủ tục báo lỗi 50002 và giao dịch bị hủy trên **cả hai** site: không có đơn DH047 ở S1, công nợ tại S0 giữ nguyên 30.900.000 đồng.



Hình 4.2: *Giao dịch phân tán: công nợ KH007 trước và sau khi lập đơn DH046; đơn vượt hạn mức của KH022 bị hủy với lỗi 50002 và không site nào bị ghi dở*

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được

Về mặt lý thuyết, đồ án đã hệ thống hóa các kiến thức nền tảng về cơ sở dữ liệu phân tán: khái niệm, kiến trúc tham chiếu, các mức trong suốt và hai bài toán thiết kế là phân mảnh và cấp phát. Trọng tâm là phân mảnh hỗn hợp: định nghĩa, hai chiến lược HV và VH cùng điều kiện để chúng tương đương, cây phân mảnh, công thức tái thiết, chứng minh tính đúng đắn, và ảnh hưởng của phân mảnh hỗn hợp đến cấp phát, xử lý truy vấn và giao dịch.

Về mặt ứng dụng, nhóm đã thiết kế CSDL phân tán cho hệ thống quản lý đơn hàng của một công ty có ba chi nhánh và một trụ sở: KhachHang theo chiến lược HV, NhanVien theo chiến lược VH, DonHang và ChiTietDonHang theo phân mảnh dẫn xuất, ChiNhanh và SanPham được nhân bản toàn phần. Thiết kế được cài đặt trên SQL Server với các view tái thiết đảm bảo trong suốt phân mảnh và vị trí, bộ truy vấn kiểm tra tự động ba quy tắc đúng đắn, các truy vấn minh họa và một giao dịch phân tán có kiểm tra hạn mức.

Qua đề tài, nhóm rút ra rằng phân mảnh hỗn hợp phát huy hiệu quả khi các nhóm người dùng khác nhau cần những phần dữ liệu khác nhau **cả theo dòng lẫn theo cột**. Lựa chọn giữa HV và VH phụ thuộc vào tiêu chí nào phân biệt các ứng dụng rõ hơn và phần dữ liệu nào không cần chia tiếp. Thiết kế phân mảnh luôn là sự đánh đổi giữa tính cục bộ của truy vấn và chi phí của cập nhật phân tán.

5.2 Hạn chế

- Thực nghiệm mô phỏng các site bằng nhiều CSDL trên cùng một máy chủ, chưa phản ánh độ trễ và sự cố mạng thực tế.
- Tần suất truy cập của các ứng dụng là giả định, chưa thu thập từ hệ thống thật.
- Chưa cài đặt cơ chế đồng bộ tự động cho các bản nhân bản, cũng như quy trình di chuyển dữ liệu khi khách hàng chuyển chi nhánh.

5.3 Hướng phát triển

- Triển khai trên nhiều máy thật nối bằng Linked Server, sử dụng MSDTC cho giao dịch phân tán, và đo thời gian phản hồi của các truy vấn cục bộ và toàn cục.
- Dùng SQL Server Replication để đồng bộ tự động các bản nhân bản ChiNhanh và SanPham.
- So sánh định lượng các phương án: phân mảnh ngang thuần, phân mảnh dọc thuần và phân mảnh hỗn hợp, trên cùng tập ứng dụng và dữ liệu lớn hơn.

TÀI LIỆU THAM KHẢO

[1] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*, 4th ed. Cham, Switzerland: Springer, 2020.

[2] S. Ceri and G. Pelagatti, *Distributed Databases: Principles and Systems*. New York, NY, USA: McGraw-Hill, 1984.

[3] Microsoft, "CREATE VIEW (Transact-SQL) – Partitioned views," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/sql/t-sql/statements/create-view-transact-sql

[4] W. T. McCormick, P. J. Schweitzer, and T. W. White, "Problem decomposition and data reorganization by a clustering technique," *Operations Research*, vol. 20, no. 5, pp. 993–1009, 1972.

[5] S. Navathe, S. Ceri, G. Wiederhold, and J. Dou, "Vertical partitioning algorithms for database design," *ACM Transactions on Database Systems*, vol. 9, no. 4, pp. 680–710, 1984.

[6] Microsoft, "Linked servers (Database Engine)," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/linked-servers/linked-servers-database-engine

[7] Microsoft, "Synonyms (Database Engine)," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/sql/relational-databases/synonyms/synonyms-database-engine